



Beaconhouse National University
13 Km Niazbeg Raiwind Road, Beacon House Society, Lahore, Pakistan

ClassSync AI

Sunbird Solutions

Version 1.0

By

Saad Mughal

F2023-009

Sheikh Muhammad Ibrahim

F2023-630

Khadijah Zahoor

F2023-956

Supervisor

Dr. Natash Ali Mian

School of Computer & IT (SCIT)

Bachelor of Science in Computer Science (2023-2027)

Table of Contents

Objective of the Project.....	3
Introduction to the Organization.....	3
Details of the Existing System.....	3
Problems in the Existing System.....	3
Proposed Solution.....	3
SystemOverview.....	3
Core Algorithmic Approach.....	3
Technical Architecture.....	4
Gantt Chart.....	4
Process Model Used.....	4
1. Clarifying User Requirements.....	4
2. Early Validation of Output.....	4
3. Risk Reduction.....	5
4. Feedback Loop.....	5
Requirements Analysis.....	5
Summary of Requirements.....	5
Functional Requirements.....	5
Data Requirements.....	5
Non-Functional Requirements.....	5
System “Shall” Statements.....	5
System Design.....	6
Use Case Model.....	6
Use Case Descriptions.....	6
UC01: Login / Register.....	6
UC02: Upload Datasets.....	7
UC03: Validate Data.....	8
UC04: Configure Constraints.....	9
UC05: Set Preferences.....	10
UC06: Generate Timetable.....	11
UC07: View Timetable.....	12
UC08: Export Timetable.....	13
UC09: Ask for Help (AI Chat).....	14
UC10: Run 'What-If' Scenarios.....	14
UC11: Explain Schedule.....	15
Sequence Diagram.....	17
UC01: Login / Register.....	17
UC02: Upload Datasets.....	18
UC03: Validate Data.....	18
UC04: Configure Constraints.....	19
Manual Configuration Sequence Diagram.....	19

AI Assisted Configuration Sequence Diagram.....	19
UC05: Set Preferences.....	20
UC06: Generate Timetable.....	20
UC07: View Timetable.....	21
UC08: Export Timetable.....	21
UC09: Ask for Help (AI Chat).....	22
UC10: Run 'What-If' Scenarios.....	23
UC11: Explain Schedule.....	23
Class Diagram.....	24
User Interface Design.....	25
Prototype Screenshots.....	25
Testing.....	29
Test Cases.....	29
Project Management.....	31
Open Issues.....	31
Project Evaluation.....	31
Learning Outcomes.....	31
Overall Experience.....	31
Conclusion.....	31

Objective of the Project

Fix the issues of manually making a time table every semester, trying to manage every section across multiple groups and semester made easier.

Introduction to the Organization

Sunbird Solutions is a start-up in the IT industry. It's a small company with 2-10 employees. The company has been focused in app and game development and currently has two games and one application live on Google Play Store. Now they have started working on their latest project ClassSync AI, a timetable scheduler for universities.

Details of the Existing System

The existing system is that the coordinator manually plots each course, with multiple sections for the students.

Problems in the Existing System

1. Time Consuming (take at least 2-2.5 weeks time frame to get to a sub optimal time table)
2. Constraints (Many Teachers are visiting faculty and as bound to certain time slots so managing each constraints for each of the teaching faculty member is very logistically difficult)
3. Butterfly Effect (Just changing one class to another time slot could cause a butterfly or ripple effect, causing double booking,clashes for students multiple classes at the exact same time)
4. Human Error (Just the sheer volume of all the different variables, and managing them manually will most definitely cause Human Error)
5. UnOptimized (Causes some unfortunate students to bare 4+ hour gaps which is just very inefficient)

Proposed Solution

SystemOverview

The proposed solution was to implement a Constraint Satisfaction Problem (CSP), as the system model uses Genetic Algorithm(GA) to efficiently allocate (Instructors, Rooms, Time Slots) while adhering to each Hard and Soft Constraints.

Core Algorithmic Approach

ClassSync AI utilizes the processing power and computes multiple generations of the time table,with a better and more efficient time table after each

generation. It uses mutation as the best version (best in terms of rewarding system), and after selection it carries on the 'genes' of the best one in that generation until the most optimal timetable is made.

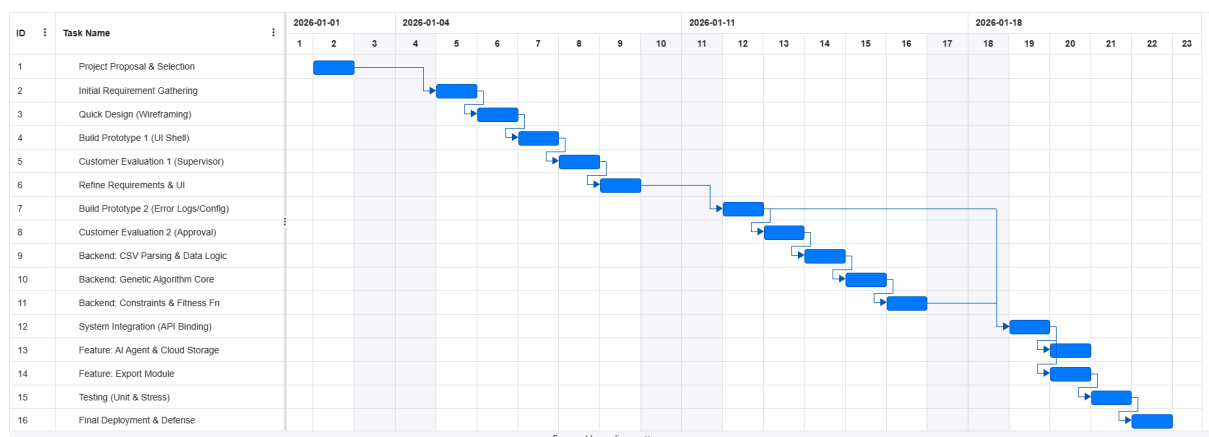
The algorithm uses as well as the rewards system, a penalty system as the penalty system acts as a fitness function, for example if one generation double books a faculty member or does follow any of the hard and soft Constraints it ends up correcting itself after the penalty and tries again. Bringing down the normal time table time from 80-100 work hours to a measly 3-4 seconds, all the while executing with such a standard that provides each of the 'Party' their own customized time table.

Technical Architecture

The Backend or the Genetic Algorithm is Built using Python, using Numpy library for high performance numerical operation and Pandas for Data supervised manipulation.

The Front is developed on Vite React Framework as it provides a very easy and intuitive way for the user to interact with the software.

Gantt Chart



Process Model Used

We selected the Rapid Prototyping Model for ClassSync AI because the scheduling requirements for the BNU were complex and difficult to visualize without a tangible interface. This model allowed us to construct a working model of the system quickly to validate requirements with our client before committing to the final development.

1. Clarifying User Requirements

The interaction between a user and an AI-based scheduler is novel. It was difficult to define exactly how the coordinator would input "Soft

Constraints" (like teacher preferences) using just text. By building a UI prototype first, we could demonstrate the "Constraint Configuration Module" and get immediate feedback on its usability.

2. Early Validation of Output

The final output is the most critical part of the system. Prototyping allowed us to manually generate sample Excel sheets and show them to the administration to ensure the format, color coding, and layout met their specific operational needs before we automated the generation logic.

3. Risk Reduction

Developing the Genetic Algorithm is computationally expensive and time-consuming. The prototyping approach allowed us to verify the frontend workflow and data input structures (CSV schemas) first. This ensured that when we began developing the backend optimization engine, the data feeding into it was already validated and stable.

4. Feedback Loop

This model facilitated a "Listen-Build-Revise" cycle. We presented the initial "Upload" and "Dashboard" screens to the supervisor, received feedback on the lack of "Error Logging," and immediately incorporated a detailed log viewer in the next prototype version.

Requirements Analysis

Summary of Requirements

It is an AI-powered university timetable scheduling tool that automates the creation of optimized academic schedules while respecting institutional constraints. It accepts academic data in standard formats, validates inputs immediately, detects inconsistencies, and stores validated data.

Scheduling logic is directed by the strict enforcement of hard constraints (teachers, rooms, students, and time slots) and the optimization of soft constraints to improve schedule quality. Users can configure rules manually as well.

The timetable is generated by a Genetic Algorithm that evolves candidate schedules, penalizes conflicts, rewards preferences, and converges on an optimal solution within defined performance limits. Depending on feasibility, the

best possible alternative can also be generated while transparently highlighting remaining conflicts. An AI explains its choices and lets you safely test changes before you download the final schedule.

Functional Requirements

This section outlines the specific behaviors and functions the ClassSync AI system must support.

Authentication & User Management

- **FR-AUTH-01 (Login):** The system shall allow users to log in using an email and password.
- **FR-AUTH-02 (RBAC):** The system shall use Role-Based Access Control with three levels: **Admin** (full access), **Coordinator** (upload and view), and **Viewer** (read-only).
- **FR-AUTH-03 (Session Management):** The system shall keep user sessions secure using JWT.
- **FR-AUTH-04 (Multi-Tenancy):** The system shall keep data separate between different universities so users only see their own institution's info.
- **FR-AUTH-05 (Subscription Check):** The system shall check if an institution has an active subscription or trial before starting the schedule generation.

Data Management (Inputs)

- **FR-DATA-01 (Dataset Upload):** The system shall accept data uploads in CSV and XLSX formats.
- **FR-DATA-02 (Validation):** The system shall immediately check uploaded files for the right format, missing info, or duplicate entries.
- **FR-DATA-03 (Storage):** The system shall store all validated data securely in cloud storage.
- **FR-DATA-04 (Versioning):** The system shall keep track of different versions of uploaded data so users can go back to older versions if needed.

Constraint Configuration

- **FR-CONF-01 (Constraint Management):** The system shall let users manage hard constraints (must-follow rules), soft constraints (preferences), and optional rules.
- **FR-CONF-02 (Timeslot Definition):** The system shall support class lengths of 60, 90, or 120 minutes.

- **FR-CONF-03 (Reset Defaults):** The system shall have a button to reset all rules back to the original factory settings.
- **FR-CONF-04 (Natural Language Config):** The system shall let users change rules by typing in plain English, which the AI then turns into official system rules.

Optimization Engine (Scheduling)

- **FR-SCHED-01 (Generation):** The system shall create a conflict-free timetable using a Genetic Algorithm that tests and improves different schedule options.
- **FR-SCHED-02 (Conflict Resolution):** The system shall treat hard constraints as non-negotiable; any schedule that breaks these rules is considered invalid.
- **FR-SCHED-03 (Soft Optimization):** The system shall try its best to follow as many soft preferences as possible to improve schedule quality.
- **FR-SCHED-04 (Performance Target):** The system shall finish generating a standard schedule (200 courses) in 60 seconds or less.
- **FR-SCHED-05 (Progress Feedback):** The system shall show live updates during the generation process, like how many attempts it has made and the current "fitness" score.

AI Agent & Assistant

- **FR-AI-01 (Interpretation):** The AI Agent shall understand user questions and find the right tools to answer them.
- **FR-AI-02 (Confirmation Loop):** The AI must ask for the user's permission before it actually changes any rules or starts a new schedule.
- **FR-AI-03 (Explanation):** The AI shall explain its logic, telling users exactly why a certain class was placed at a specific time.
- **FR-AI-04 (What-If Scenarios):** The system shall have a "sandbox" mode to test "what-if" changes safely without messing up the live schedule.
- **FR-AI-05 (Dataset Diagnosis):** The AI shall scan data for logic errors (like more class hours than available rooms) and suggest how to fix them.

Export & Visualization (Outputs)

- **FR-EXP-01 (Timetable Views):** The system shall show schedules in interactive grids that can be filtered by student, teacher, or room.
- **FR-EXP-02 (File Export):** The system shall let users download the final timetable in formats like XLSX, PDF, and PNG.

Data Requirements

This section defines the specific data structures and formats required for the system to function.

Entity Definitions

- **DR-01 (Courses Entity):** The system shall store Course data including:
 - **Course ID:** Unique Alphanumeric Identifier (e.g., CS101).
 - **Course Name:** String.
 - **Credit Hours:** Integer (defining contact hours).
 - **Instructor ID:** Foreign Key linking to Teachers.
 - **Section:** Alphanumeric (e.g., Section A).
 - **Lab Requirement:** Boolean (Yes/No).
 - **Duration:** Integer (minutes per session).
- **DR-02 (Rooms Entity):** The system shall store Room data including:
 - **Room ID:** Unique Identifier.
 - **Capacity:** Integer (Maximum student count).
 - **Type:** Enumerated (Lecture Hall, Laboratory, Seminar Room).
 - **Equipment:** List (Projector, Whiteboard, PCs).
- **DR-03 (Teachers Entity):** The system shall store Teacher data including:
 - **Teacher ID:** Unique Identifier.
 - **Name:** String.
 - **Available Slots:** Array of Time/Day pairs.
 - **Max Load:** Integer (Maximum hours per week).

Data Integrity & Format

- **DR-04 (Input Format):** The system shall exclusively accept data inputs in .csv (Comma Separated Values) or .xlsx (Excel Workbook) formats.
- **DR-05 (Data Relationships):** The system shall enforce referential integrity; a Course cannot be assigned to a non-existent Instructor ID or Room ID.
- **DR-06 (Data Retention):** The system shall retain generated schedule data for a minimum of 1 academic year unless manually deleted by the administrator.

Non-Functional Requirements

These requirements define the quality attributes, user experience, and operational standards of the system.

Look and Feel Requirements

- **NFR-LF-01 (Interface Style):** The web interface shall utilize the Vite React framework to provide a modern, responsive, and clean aesthetic suitable for administrative use.
- **NFR-LF-02 (Color Coding):** The generated timetable grids shall use distinct color coding: Green for valid slots, Red for conflicts, Blue for Labs, and Purple for administrative breaks.
- **NFR-LF-03 (Dashboard Visualization):** The analytics dashboard shall present data using professional charting libraries to display "Fitness Evolution" and "Resource Utilization" clearly.

Usability Requirements

- **NFR-USE-01 (Accessibility):** The web interface shall adhere to basic accessibility standards (WCAG 2.1 Level A), ensuring high contrast and clear labeling.
- **NFR-USE-02 (Clarity):** Error messages and AI explanations must be presented in plain, non-technical language (e.g., "Room is full" instead of "Capacity Exception").
- **NFR-USE-03 (Learning Curve):** The system shall be designed such that a new "Coordinator" level user can upload a dataset and generate a schedule within 15 minutes of first use.

Performance Requirements

- **NFR-PERF-01 (Generation Speed):** The optimization engine must produce a feasible schedule for a standard dataset (approx. 200 courses) within 60 seconds.
- **NFR-PERF-02 (UI Latency):** Dashboard screens and standard interactions must load in ≤ 2 seconds under normal network conditions.
- **NFR-PERF-03 (AI Latency):** The AI Agent must return a response to standard queries within 2 to 5 seconds.

Operational Requirements

- **NFR-OP-01 (Availability):** The SaaS platform shall target a monthly availability of 99.5% during business hours.
- **NFR-OP-02 (Error Handling):** The system shall degrade gracefully; failed optimization runs must not corrupt existing data.
- **NFR-OP-03 (Auto-Save):** User configurations and constraints shall be auto-saved every 5 minutes to prevent data loss.

Maintainability and Portability Requirements

- **NFR-MP-01 (Modularity):** The system shall maintain strict separation between the Frontend, Backend API, and Genetic Engine.
- **NFR-MP-02 (Containerization):** The backend architecture shall be containerized using Docker to ensure deployment consistency across environments.

- **NFR-MP-03 (Code Quality):** The optimization core shall utilize best programming practices..

Security Requirements

- **NFR-SEC-01 (Encryption):** All data in transit must be encrypted via HTTPS/TLS 1.2+.
- **NFR-SEC-02 (Password Storage):** User passwords must be salted and hashed using strong algorithms (e.g., bcrypt or Argon2).
- **NFR-SEC-03 (Audit Logging):** All critical actions (constraint modifications, exports) must be logged for audit purposes.
- **NFR-SEC-04 (Role-Based Access):** The system must enforce strict isolation where "Viewers" cannot trigger schedule generation.

Cultural and Political Requirements

- **NFR-CP-01 (Institutional Hierarchy):** The system shall accommodate hierarchical scheduling priority (e.g., "Dean's Preferences" treated as high priority).
- **NFR-CP-02 (Workload Fairness):** The optimization logic shall prioritize equitable distribution of teaching hours to prevent friction regarding workload imbalance.

Legal Requirements

- **NFR-LEG-01 (Data Privacy):** Institution data must be logically isolated and must not be used to train public LLM models.
- **NFR-LEG-02 (Compliance):** The system shall allow administrators to permanently delete all institutional data upon subscription cancellation.

System “Shall” Statements

1. Input and Data Handling

- **SYS-REQ-01:** The system shall accept input data via CSV files for "Courses" (Course ID, Name, Instructor, Section) and "Rooms" (Room ID, Capacity, Type).
- **SYS-REQ-02:** The system shall validate all uploaded CSV files to ensure correct column headers and data types before processing.
- **SYS-REQ-03:** The system shall parse the input data to map unique identifiers to courses, instructors, and rooms for algorithmic processing.

2. Algorithmic Processing (The Genetic Algorithm)

- **SYS-REQ-04:** The system shall utilize a Genetic Algorithm (GA) to generate a population of candidate timetables.
- **SYS-REQ-05:** The system shall calculate a "Fitness Score" for every generated timetable based on a weighted sum of constraint violations.
- **SYS-REQ-06:** The system shall implement an Instructor-Aware Crossover mechanism to exchange timetable data without breaking instructor assignments.
- **SYS-REQ-07:** The system shall apply Adaptive Mutation strategies, increasing mutation rates for low-fitness schedules and decreasing them for high-fitness schedules.
- **SYS-REQ-08:** The system shall employ a Fallback Placement Mechanism using a greedy heuristic to assign difficult-to-place courses if genetic operations fail.

3. Constraint Enforcement

- **SYS-REQ-09:** The system shall strictly enforce Instructor Non-Overlap, ensuring no instructor is assigned to two locations at the same time.
- **SYS-REQ-10:** The system shall strictly enforce Room Capacity, ensuring student enrollment does not exceed the room's limit.
- **SYS-REQ-11:** The system shall validate Room Type Compatibility, ensuring laboratory courses are only assigned to rooms designated as laboratories.
- **SYS-REQ-12:** The system shall optimize for Section Isolation, minimizing time overlaps between courses in the same student section.

4. Output and Reporting

- **SYS-REQ-13:** The system shall export the final optimized timetable as an Excel workbook (.xlsx) using the OpenPyXL library.
- **SYS-REQ-14:** The system shall generate three distinct worksheet views in the Excel output: Section-wise, Instructor-wise, and Room-wise.
- **SYS-REQ-15:** The system shall apply specific Color Coding to cells (e.g., Blue for Labs, Purple for VC Breaks) to distinguish session types.
- **SYS-REQ-16:** The system shall display a real-time Analytics Dashboard showing the evolution of the fitness score across generations.
- **SYS-REQ-17:** The system shall log all detected conflicts and resolution steps to a text file for administrative review.

5. Performance and Operations

- **SYS-REQ-18:** The system shall complete the schedule generation process for a standard dataset (approx. 45 courses) in less than 15 seconds.

- **SYS-REQ-19:** The system shall utilize multithreading to perform parallel fitness evaluations during algorithm execution.

4.1 Dataset Upload & Management Requirements

- **REQ-F-01:** The system shall allow authorized users to upload academic datasets in CSV or XLSX formats for Courses, Teachers, Rooms, and Sections.
- **REQ-F-02:** The system shall automatically validate uploaded files against a predefined schema to detect missing fields or incorrect data.
- **REQ-F-03:** The system shall utilize the AI Agent to identify dataset inconsistencies (e.g., "Teacher assigned to 0 courses") and suggest corrections.
- **REQ-F-04:** The system shall securely store valid datasets in cloud storage with unique identifiers for each institution.
- **REQ-F-05:** The system shall maintain a version history of uploaded datasets, allowing users to revert to previous versions.

4.2 Constraint Configuration Requirements

- **REQ-F-06:** The system shall provide a user interface for modifying both Hard Constraints (mandatory) and Soft Constraints (preferences).
- **REQ-F-07:** The system shall allow users to toggle "Optional Constraints" on or off, such as enforcing strict Room Capacity limits.
- **REQ-F-08:** The system shall support configurable timeslot durations of 60, 90, or 120-minute blocks.
- **REQ-F-09:** The system shall provide a "Reset to Defaults" function to revert all configurations to original system settings.
- **REQ-F-10:** The system shall require user confirmation before the AI Agent applies automated modifications to active constraints.

4.3 Optimization Engine Requirements

- **REQ-F-11:** The system shall generate a conflict-free timetable that adheres to 100% of the active Hard Constraints.
- **REQ-F-12:** The system shall use a weighted scoring algorithm to maximize adherence to Soft Constraints.
- **REQ-F-13:** The system shall complete the generation process within 60 seconds for a standard dataset (200 courses, 80 teachers, 60 rooms).
- **REQ-F-14:** The system shall automatically detect remaining conflicts after generation and report them in a detailed log.

4.4 AI Agent Requirements

- **REQ-F-16:** The system shall include an AI Agent that interprets natural language instructions and converts them into system operations.
- **REQ-F-18:** The AI Agent shall diagnose dataset inconsistencies and explain them to the user in plain language.
- **REQ-F-19:** The AI Agent shall provide explanations for specific scheduling decisions upon user request.
- **REQ-F-20:** The system shall allow the AI Agent to run "What-If" simulations to compare how constraint changes affect the final schedule.
- **REQ-F-21:** The system shall use GPT-4.1 as the primary model for reasoning, with Gemini 2.0 as the fallback model.

4.5 Export & Visualization Requirements

- **REQ-F-22:** The system shall export timetables in formats including XLSX, CSV, PNG, PDF, JSON, and ZIP.
- **REQ-F-23:** The system shall generate filtered views for specific entities: Per-Section, Per-Room, and Per-Teacher.
- **REQ-F-24:** The system shall provide an interactive, color-coded visual viewer for the timetable in the web interface.

4.6 Authentication & Access Requirements

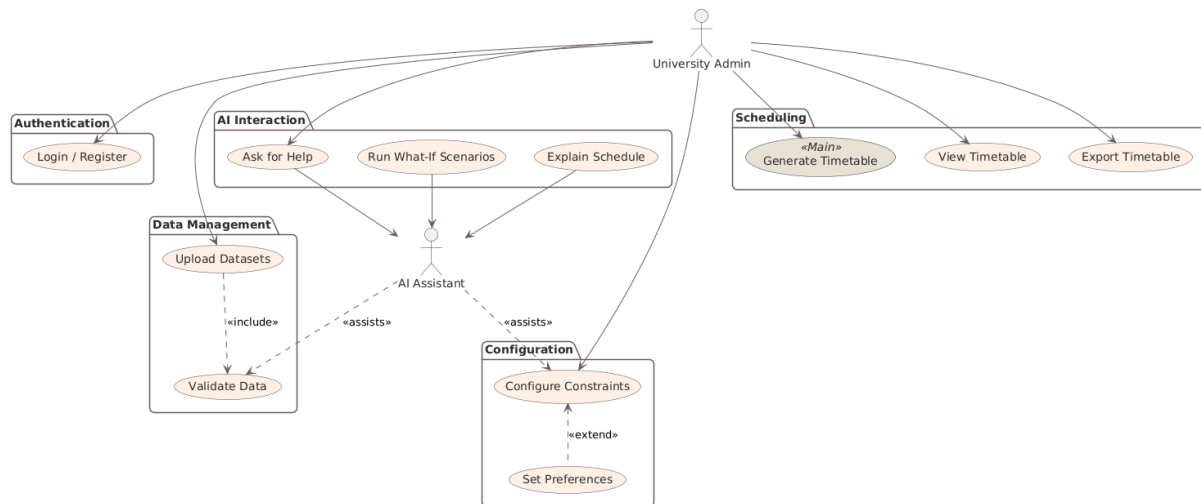
- **REQ-F-25:** The system shall authenticate users using secure Email and Password credentials.
- **REQ-F-26:** The system shall enforce Role-Based-Access Control for Admins, Coordinators, and Viewers.
- **REQ-F-27:** The system shall isolate data at the institution level to ensure no data leakage between different university tenants.

4.7 Subscription & Billing Requirements

- **REQ-F-28:** The system shall support a "Free Trial" mode for new institutions.
- **REQ-F-29:** The system shall validate the institution's active subscription status before starting the generation process.

System Design

Use Case Model



Use Case Descriptions

UC01: Login / Register

Number	1	
Name	Login / Register	
Summary	The user authenticates with the system to access the dashboard and institution-specific data.	
Priority	5	
Preconditions	1	The user has a valid email and password (for Login) or a valid email (for Register).
	2	The system database is accessible.
Postconditions	1	The user is authenticated.
	2	A secure session (JWT) is established.
	3	The user is redirected to the Dashboard.
Primary Actor(s)	University Admin	
Secondary Actor(s)	System	
Trigger	The user navigates to the landing page and selects "Login" or "Register".	
Main Scenario	Step	Action

	1	The user enters email and password on the login screen.
	2	The user clicks the "Login" button.
	3	The system validates the format of the input credentials.
	4	The system verifies credentials against the database.
	5	The system generates a JWT token for the session.
	6	The system redirects the user to the main dashboard.
Extensions	Step	Branching Action
		Invalid Email/Password Format:
	3a	The system displays a validation error message.
		The user re-enters credentials.
		Authentication Failed:
	4a	The system displays an "Invalid credentials" error.
		The user retries or selects "Forgot Password".
Open Issues		Passwords must be stored using bcrypt/Argon2 hashing (NFR-SEC-02).

UC02: Upload Datasets

Number	2	
Name	Upload Datasets	
Summary	The admin uploads academic data files (CSV/XLSX) required for scheduling.	
Priority	2	
Preconditions	The user is logged in (UC01)	
Postconditions	1	Files are stored in S3-compatible storage
	2	Dataset metadata is recorded in the database.
Primary Actor(s)	University Admin	
Secondary Actor(s)	System, AI Assistant	
Trigger	The user selects "Upload Data" from the dashboard.	
Main Scenario	Step	Action

	1	The user selects the file type to upload (Courses, Teachers, Rooms, or Sections)
	2	The user drags and drops the file or selects it via the file browser.
	3	The system receives the file and initiates the upload process.
	4	The system includes validate data (UC03) to check file integrity and schema.
	5	The system saves the valid file to cloud storage.
	6	The system updates the dataset status indicator on the dashboard.
Extensions	Step	Branching Action
	4a	Validation Fails:
		The system rejects the upload
		The system using AI Assistant displays specific error details
		The user corrects the files and re-uploads.
Open Issues		Must support CSV and XLSX formats (FR-DATA-01)

UC03: Validate Data

Number	3	
Name	Validate Data	
Summary	The system checks uploaded datasets for schema compliance, duplicates, and logical errors.	
Priority	3	
Preconditions	A file upload process (UC02) has been initiated	
Postconditions	1	The dataset is marked as "Valid" or "Invalid".
	2	Error report is generated if invalid.
Primary Actor(s)	System	
Secondary Actor(s)	AI Assistant	
Trigger	Automatically triggered by upload datasets (UC02)	

Main Scenario	Step	Action
	1	The system parses the uploaded file.
	2	The system checks for mandatory columns and correct data types.
	3	The system checks for duplicate entries (e.g., duplicate course codes).
	4	AI Assistant analyzes data for logical inconsistencies (e.g., "Room capacity < Course enrollment").
	5	The system returns the validation status to the user.
Extensions	Step	Branching Action
Open Issues		Validation must complete in ≤ 3 seconds for standard files (NFR-OERF-02).

UC04: Configure Constraints

Number	4	
Name	Configure Constraints	
Summary	The Admin defines hard constraints and soft constraints for the schedule.	
Priority	4	
Preconditions	The user is logged in.	
Postconditions	Constraint profile is updated in the database.	
Primary Actor(s)	University Admin	
Secondary Actor(s)	AI Assistant, System	
Trigger	The user navigates to the "Constraints" panel or writes a prompt to the AI Assistant.	
Main Scenario	Step	Action
	1	The user navigates to the constraints configuration screen.
	2	The system fetches and displays current hard and soft constraints.

	3	The user modifies constraint parameters.
	4	The user clicks "Save".
	5	The system persists the new configuration to the database.
Extensions	Step	Branching Action
	1a	The user types a prompt in the AI chat.
	2a	The AI assistant interprets the intent and maps it to a tool call (e.g., update constraints)
	3a	The AI assistant asks for confirmation "I will block all labs on Fridays, can you confirm?".
	4a	The user confirms.
	5a	The system executes the update and confirms success.
Open Issues		All constraint modifications via AI require explicit user confirmation (FR-AI-02).

UC05: Set Preferences

Number	5	
Name	Set Preferences	
Summary	The admin fine-tunes soft constraints and global scheduling preferences.	
Priority	7	
Preconditions	The user is accessing the constraints panel.	
Postconditions	Preference weights and settings are updated.	
Primary Actor(s)	University Admin	
Secondary Actor(s)	System	
Trigger	The user interacts with specific "Preferences" or "Soft Constraints" controls.	
Main Scenario	Step	Action
	1	The user extends the configuration process (Extends UC04).
	2	The user adjusts sliders for soft constraint weights.

	3	The user selects global settings (e.g., "Timeslot duration: 90 mins").
	4	The user saves changes.
	5	The system applies these preferences to the scoring algorithm.
Extensions	Step	Branching Action
Open Issues		Input data can only be in .csv format

UC06: Generate Timetable

Number	6	
Name	Generate Timetable	
Summary	The system runs the Genetic Algorithm optimization engine to produce a conflict-free timetable.	
Priority	1	
Preconditions	1	Datasets (Courses, Teachers, Rooms) are uploaded and valid.
	2	Subscription status is active.
Postconditions	1	A new timetable is generated and saved to the database.
	2	The user is presented with the results.
Primary Actor(s)	University Admin	
Secondary Actor(s)	System, Optimization Engine	
Trigger	The user clicks the "Generate Timetable" button.	
Main Scenario	Step	Action
	1	The user clicks "Generate Timetable".
	2	The system (API) fetches active datasets and constraints from the database.
	3	The system validates hard constraints against the data.
	4	The optimization engine initiates the genetic algorithm.
	5	The optimization engine iteratively improves the schedule to satisfy hard constraints and maximize soft

		constraints.
	6	The system provides real-time progress feedback to the user.
	7	The optimization engine returns the best schedule found.
	8	The system saves the timetable entries to the database.
	9	The system displays the "Generation Complete" message and displays the timetable view.
Extensions	Step	Branching Action
	7a	Generation timeout / Failure:
	1	The system detects timeout (> 60 sec) or failure to converge.
	2	The system saves the best partial solution or logs the error
	3	The system notifies the user of the failure and suggests quick fixes like relaxing constraints.
Open Issues		Must complete within 60 seconds for standard datasets (NFR-PERF-01).

UC07: View Timetable

Number	7	
Name	View Timetable	
Summary	The admin views the generated schedule in various formats.	
Priority	6	
Preconditions	A timetable has been generated (UC06).	
Postconditions	Schedule data is visualized.	
Primary Actor(s)	University Admin	
Secondary Actor(s)	System	
Trigger	The user navigates to the "Timetable" page.	
Main Scenario	Step	Action
	1	The user selects a view mode (By Teacher, By Room, or

		By Section).
	2	The system fetches the relevant schedule entries from the database.
	3	The system renders the interactive color-coded grid.
	4	The user clicks on a specific session to view details (Course Name, Time, Room).
Extensions	Step	Branching Action
	4a	Filter Data:
	1	The user applies a filter (e.g., "Show only Computer Science Labs").
	2	The system then updates the view to match the filter.
Open Issues		UI must remain responsive (< 2 sec load time).

UC08: Export Timetable

Number	8	
Name	Export Timetable	
Summary	The admin downloads the timetable in a portable format for distribution.	
Priority	8	
Preconditions	A timetable exists.	
Postconditions	An output of the timetable is downloaded to the user's device.	
Primary Actor(s)	University Admin	
Secondary Actor(s)	System	
Trigger	The user clicks the "Export" button on the timetable view page.	
Main Scenario	Step	Action
	1	Click the "Export" button.
	2	The user selects the desired format.
	3	The system generates the file based on the current schedule data.

	4	The system provides a download link.
	5	The user downloads the file.
Extensions	Step	Branching Action
Open Issues		Supported formats: XLSX, CSV, PNG, PDF, JSON, ZIP.

UC09: Ask for Help (AI Chat)

Number	9	
Name	Ask for Help (AI Chat)	
Summary	The user interacts with the AI assistant using prompts to get help, explanations, or navigation assistance.	
Priority	11	
Preconditions	The user is logged in.	
Postconditions	AI provides a text response or performs an action.	
Primary Actor(s)	University Admin	
Secondary Actor(s)	AI Assistant	
Trigger	The user enters a prompt into the AI chat panel.	
Main Scenario	Step	Action
	1	The user types a query in the form of a prompt.
	2	The AI assistant analyzes the text to determine the intent of the user.
	3	The AI assistant retrieves relevant documentation or context.
	4	The AI assistant gives an answer.
Extensions	Step	Branching Action
Open Issues		AI must use GPT 4.1 or Gemini 2.0 (FR-AI-21)

UC10: Run 'What-If' Scenarios

Number	10
---------------	----

Name	Run "What-if" Scenarios	
Summary	The admin simulates changes to constraints to see the potential impact on the schedule without overwriting the master timetable.	
Priority	10	
Preconditions	A baseline schedule exists.	
Postconditions	A simulation report is displayed comparing the baseline vs. the new scenario.	
Primary Actor(s)	University Admin	
Secondary Actor(s)	AI Assistant, System	
Trigger	The user asks AI "What happens if..." or selects "Simulate Scenario" in UI.	
Main Scenario	Step	Action
	1	The user proposes a change.
	2	The AI assistant interprets this as a temporary constraint set.
	3	The system runs a shadow instance of generate timetable (UC06) using the temporary constraints.
	4	The system compares the shadow schedule with the current master schedule.
	5	AI assistant summarizes the impact.
Extensions	Step	Branching Action
Open Issues		

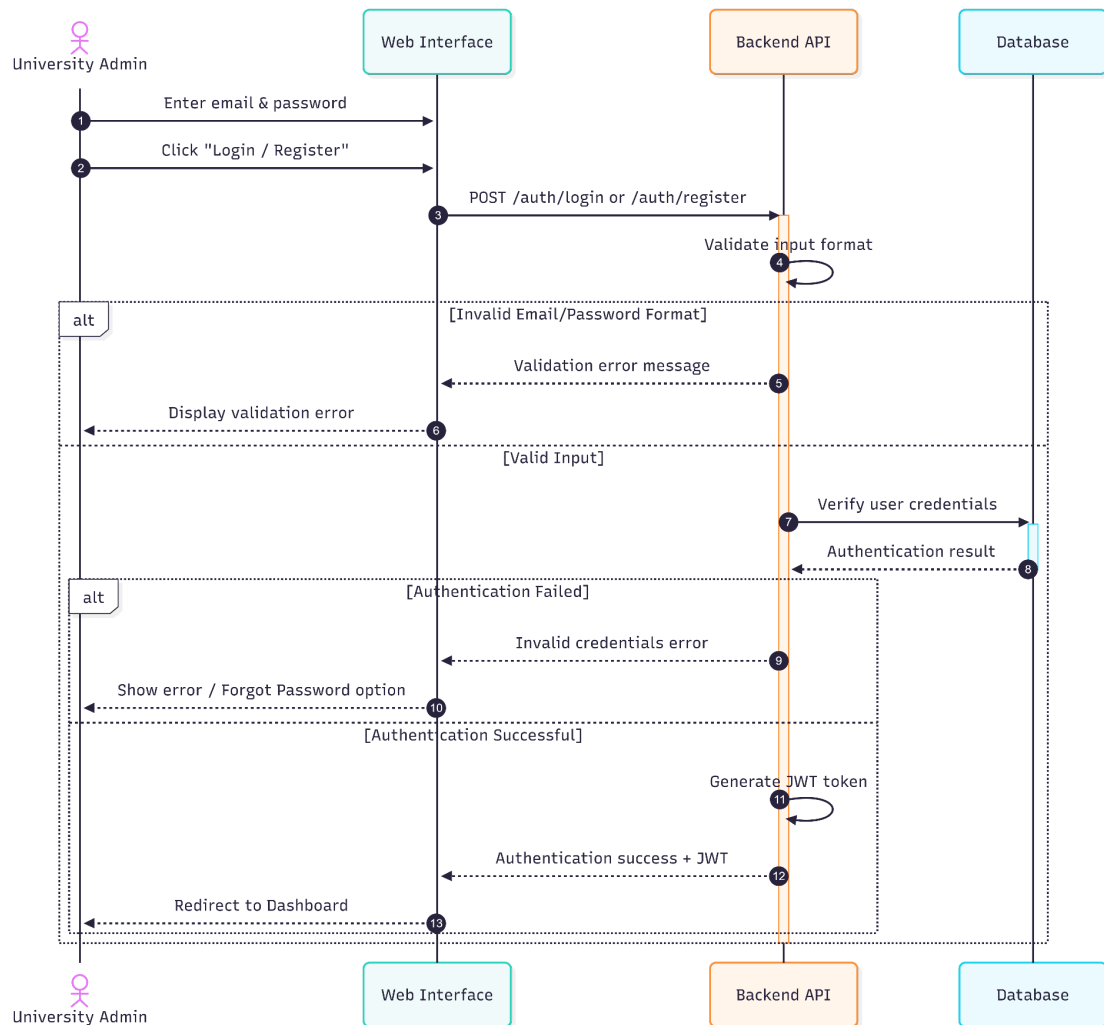
UC11: Explain Schedule

Number	11
Name	Explain Schedule
Summary	The AI assistant explains why a specific class was scheduled at a specific time/room.
Priority	9

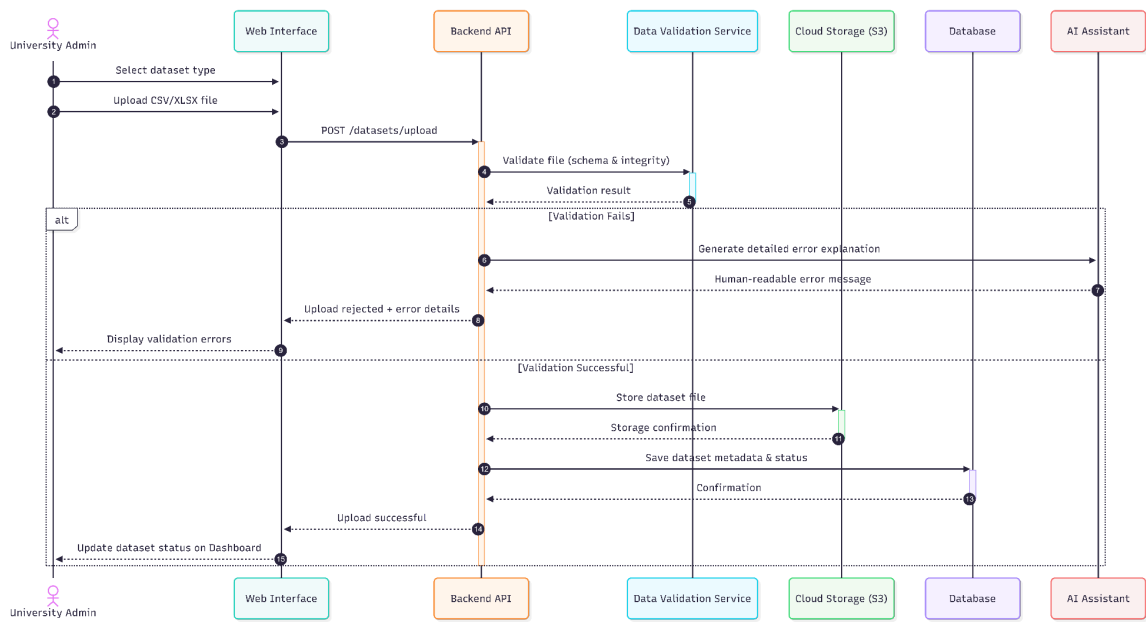
Preconditions	A time is visible (UC07).	
Postconditions	Explanation is displayed to the user.	
Primary Actor(s)	University Admin	
Secondary Actor(s)	AI Assistant	
Trigger	The user clicks "Explain this slot" or asks via prompt "Why is [course] here?"	
Main Scenario	Step	Action
	1	The user selects a scheduled session and requests an explanation.
	2	AI Assistant
	3	User Will Upload the csv file
	4	User Will Select if it is Room or Courses dataset
Extensions	Step	Branching Action
Open Issues		Requires traceability of GA decision-making or post-hoc analysis.

Sequence Diagram

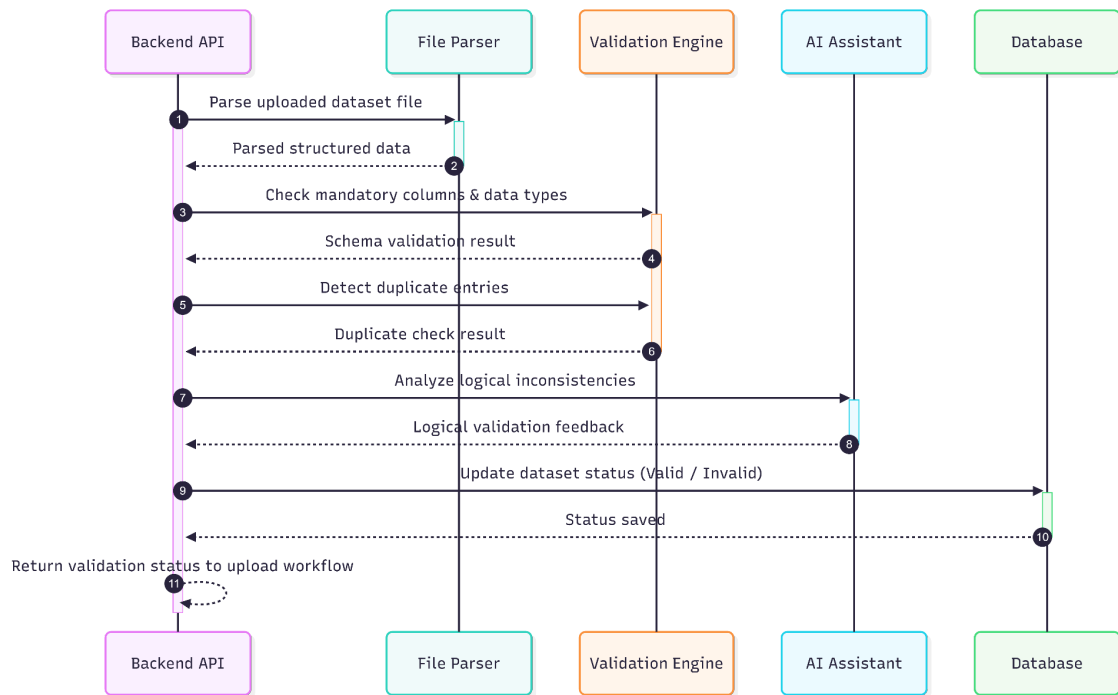
UC01: Login / Register



UC02: Upload Datasets

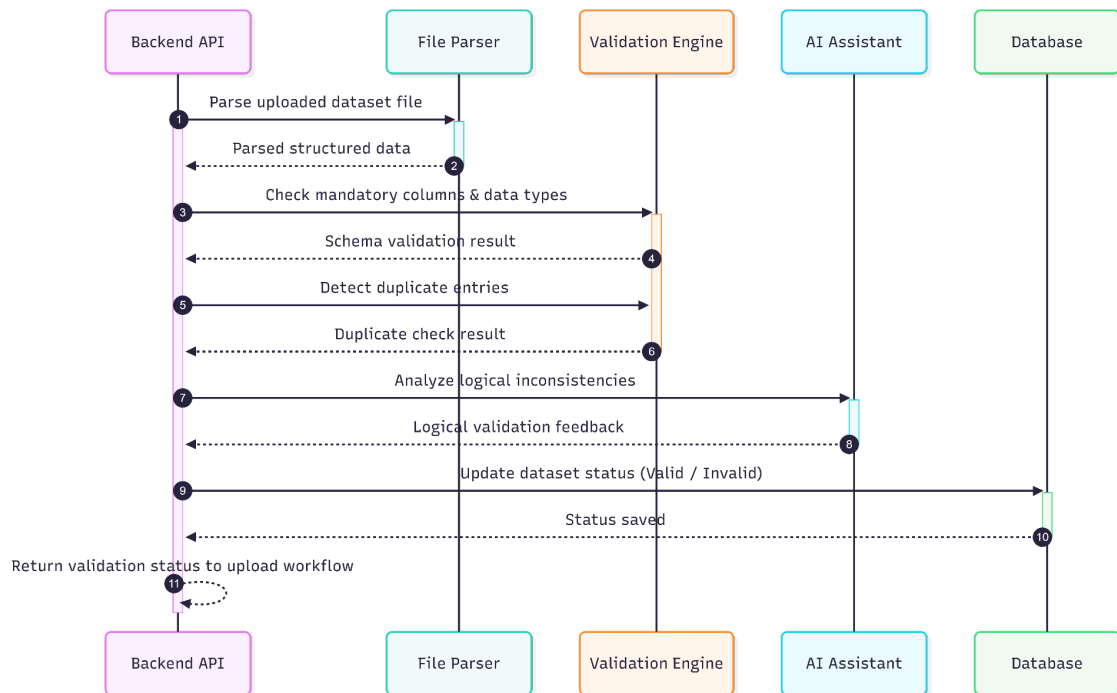


UC03: Validate Data

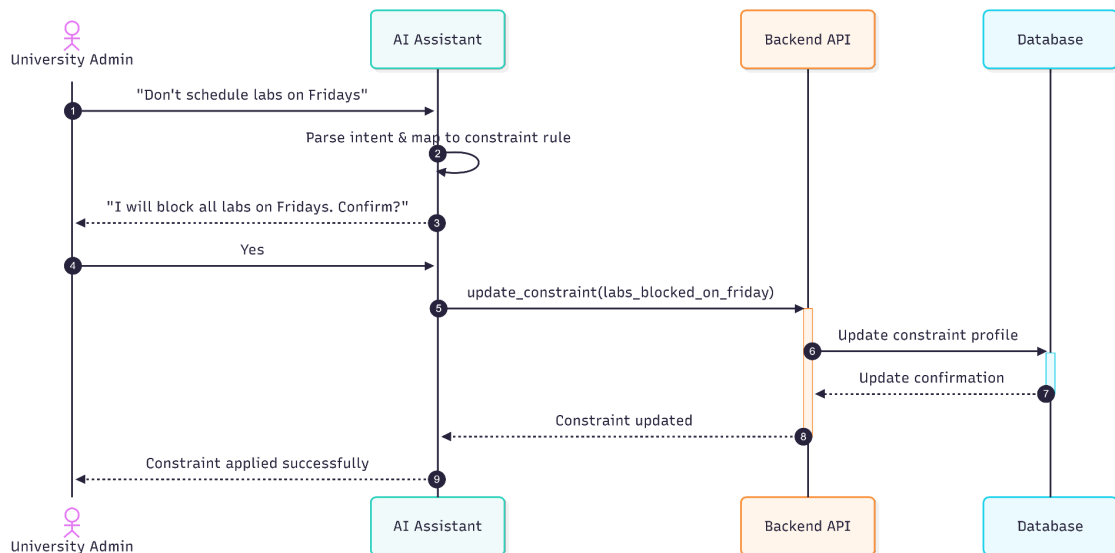


UC04: Configure Constraints

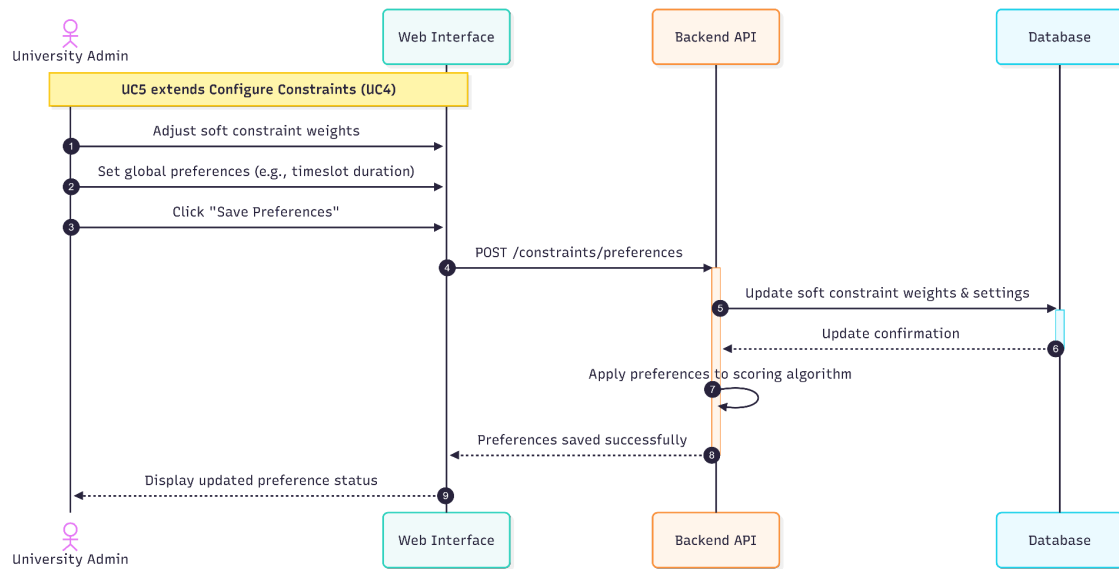
Manual Configuration Sequence Diagram



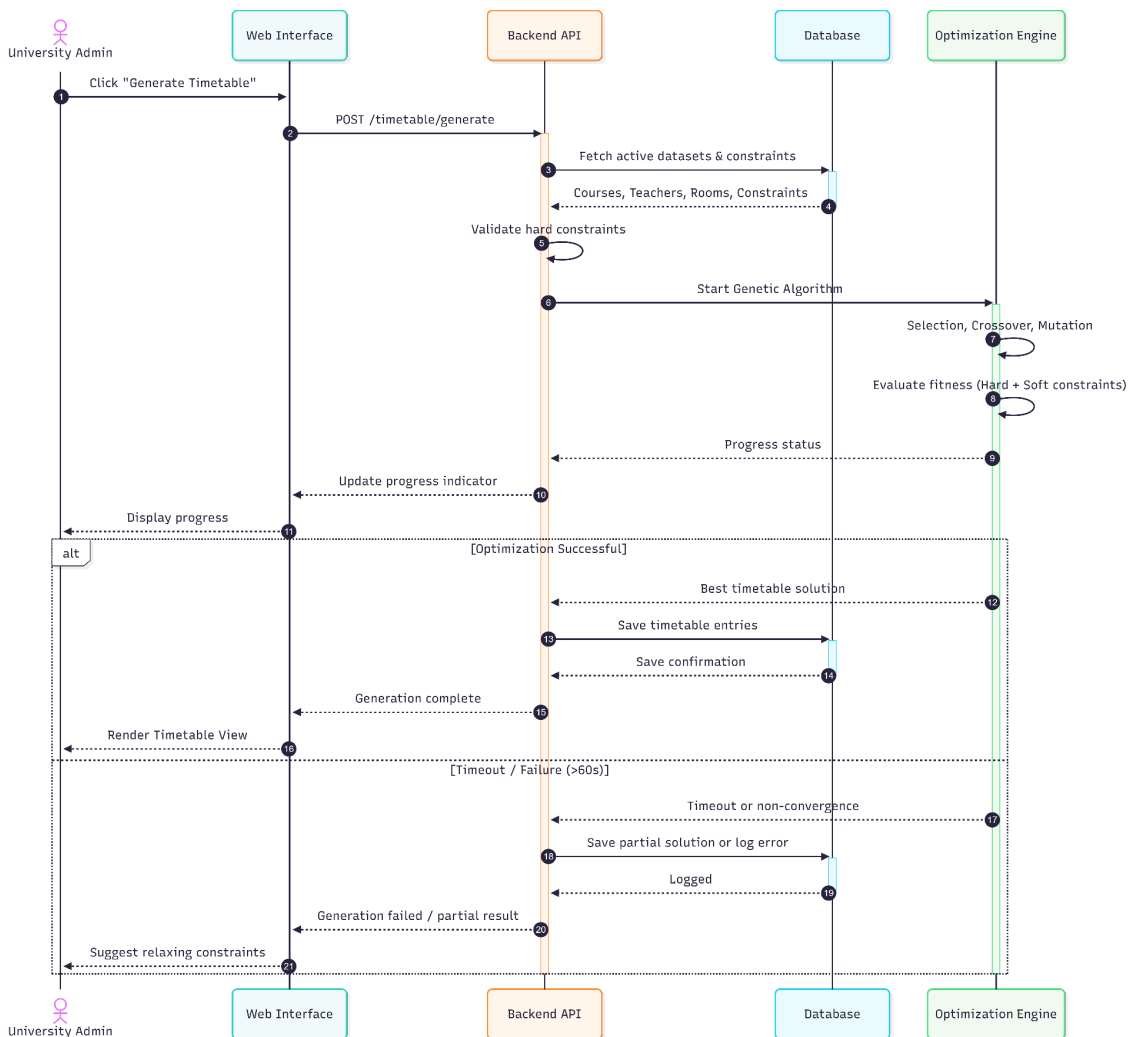
AI Assisted Configuration Sequence Diagram



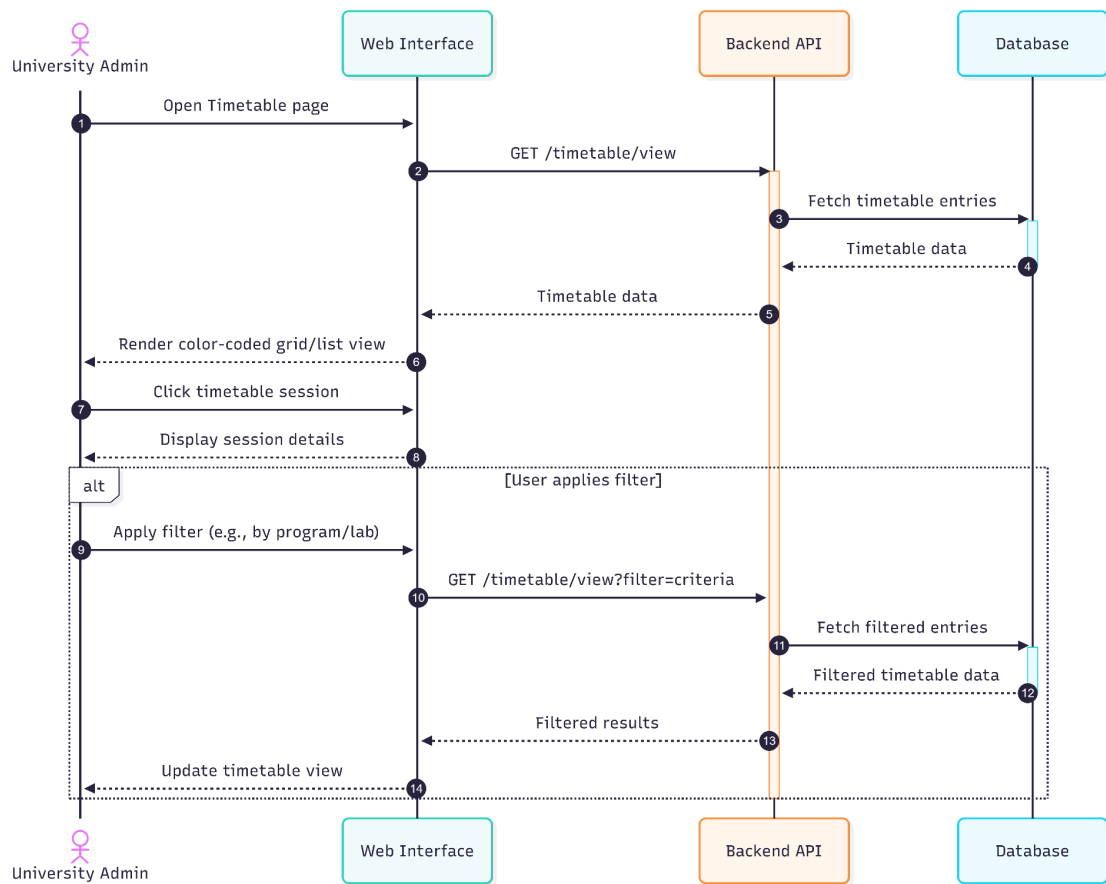
UC05: Set Preferences



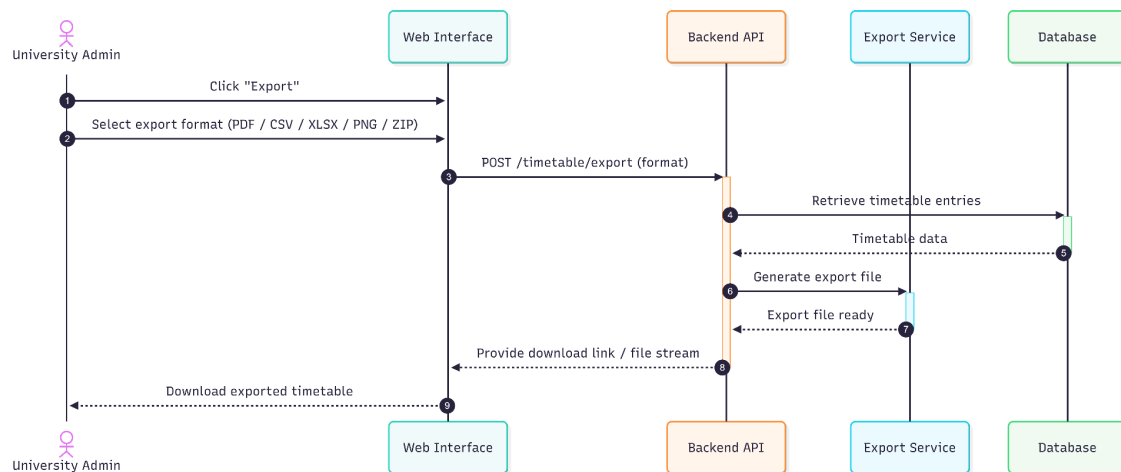
UC06: Generate Timetable



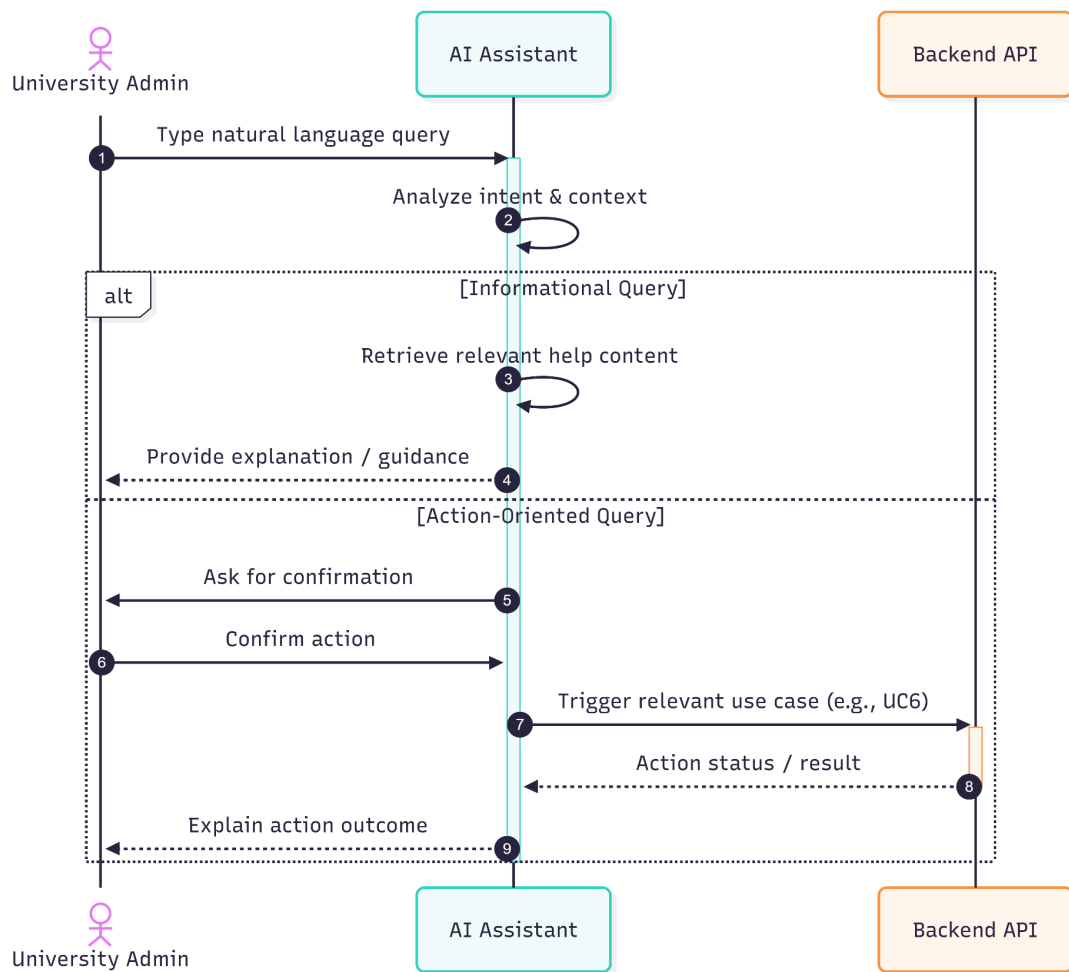
UC07: View Timetable



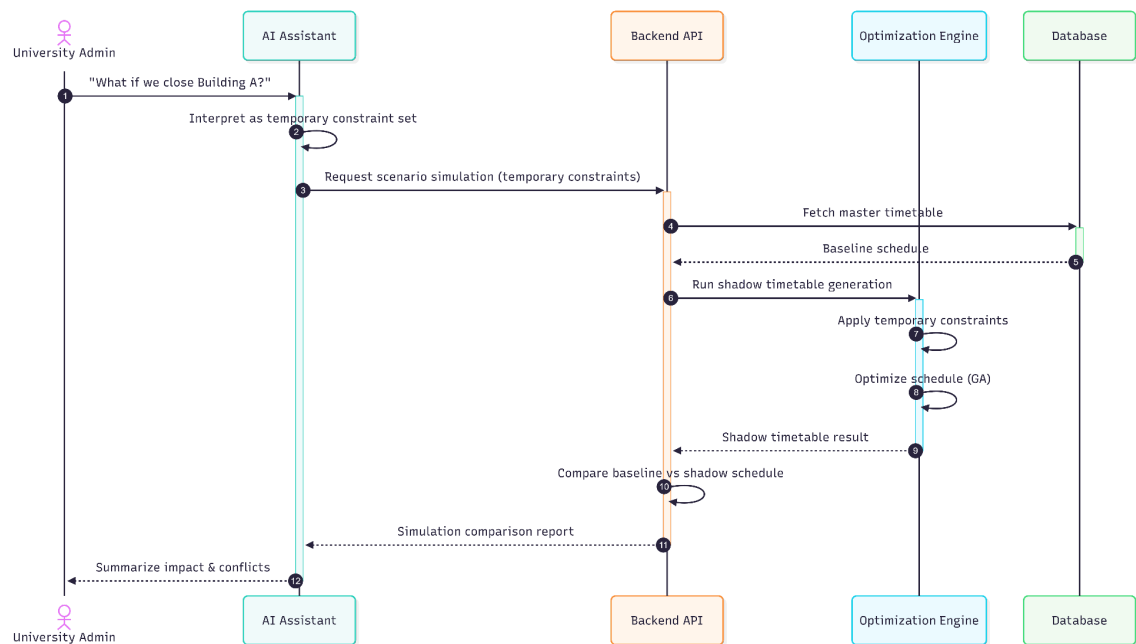
UC08: Export Timetable



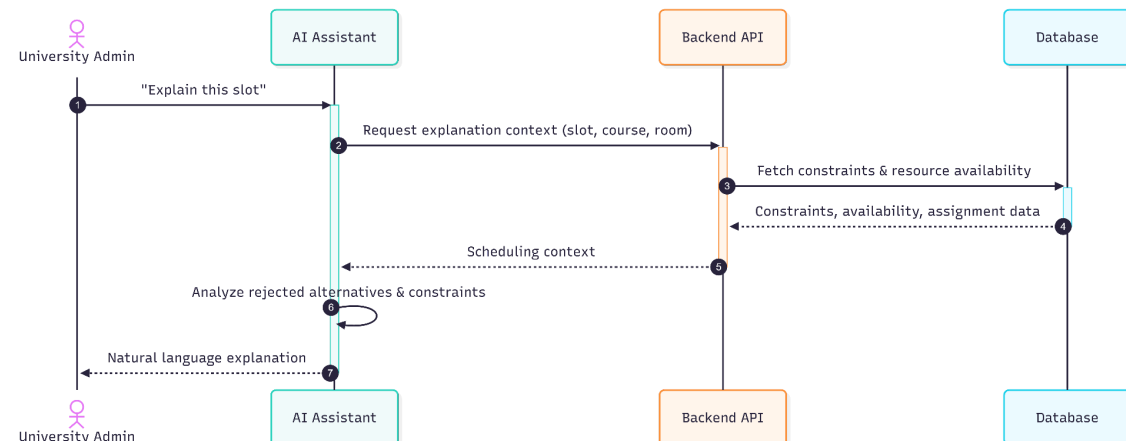
UC09: Ask for Help (AI Chat)



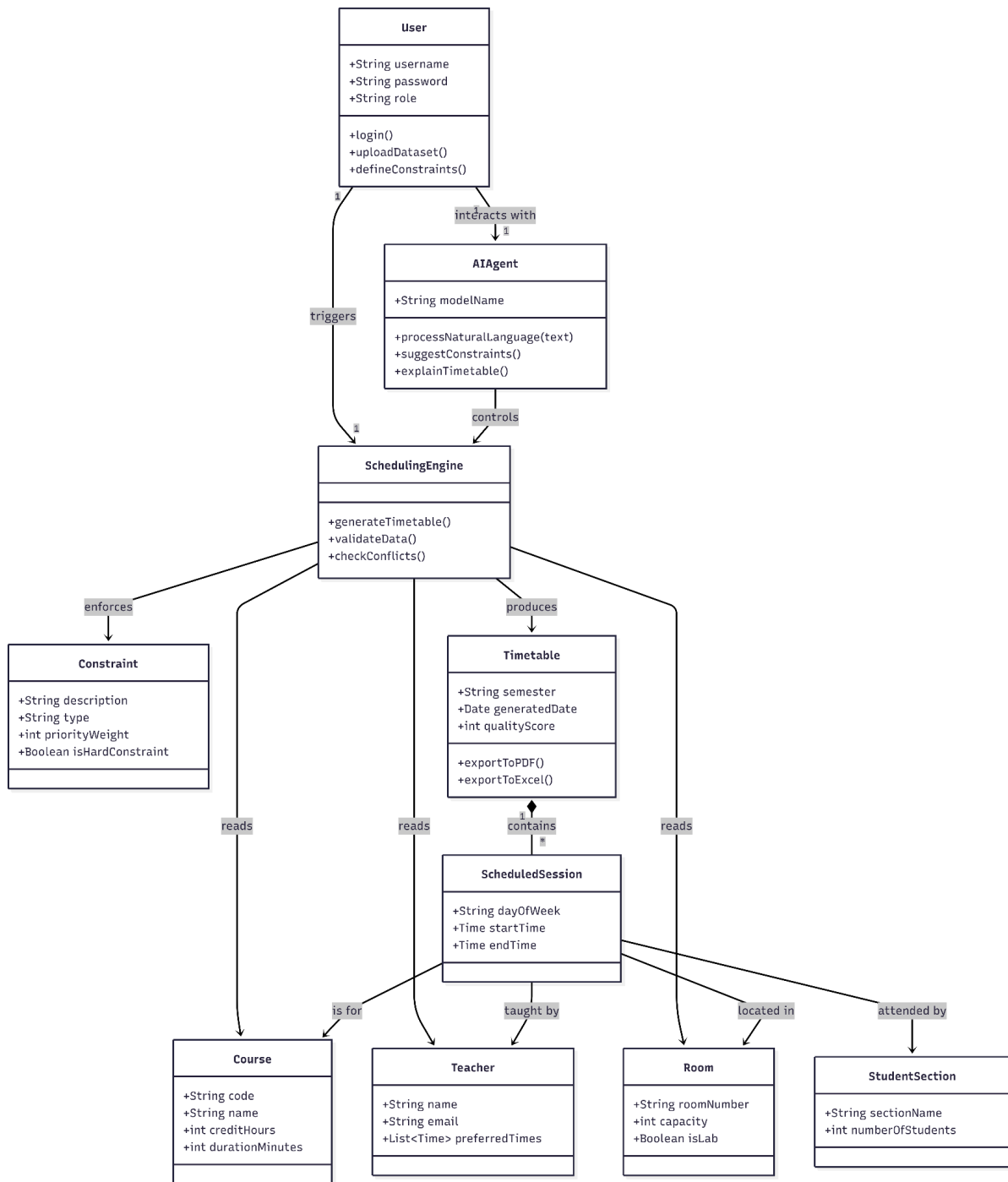
UC10: Run 'What-If' Scenarios



UC11: Explain Schedule



Class Diagram

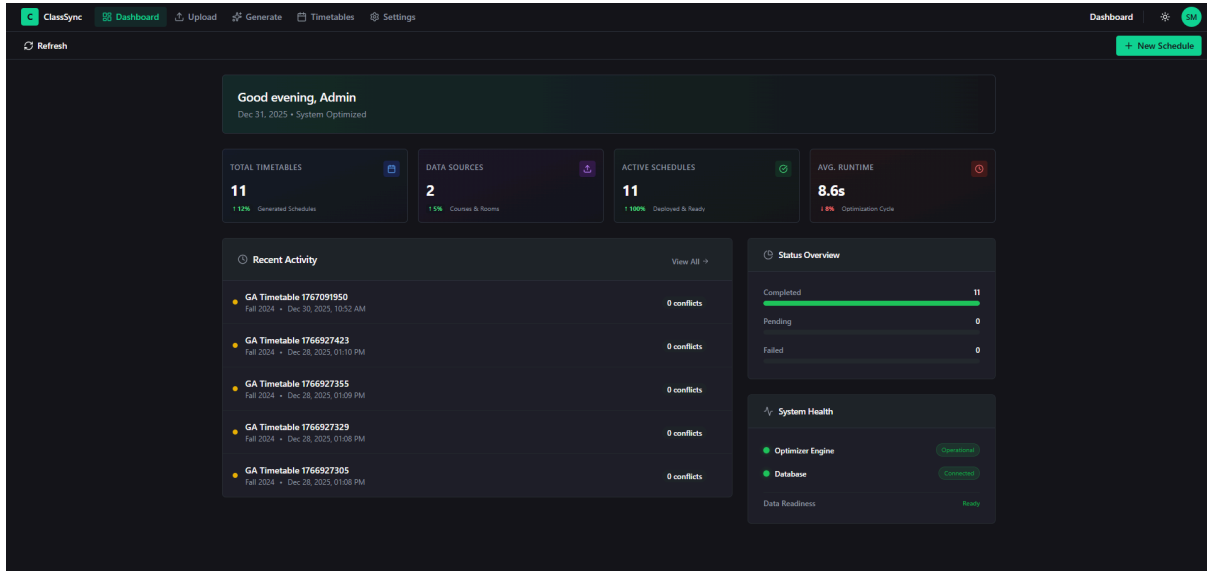


User Interface Design

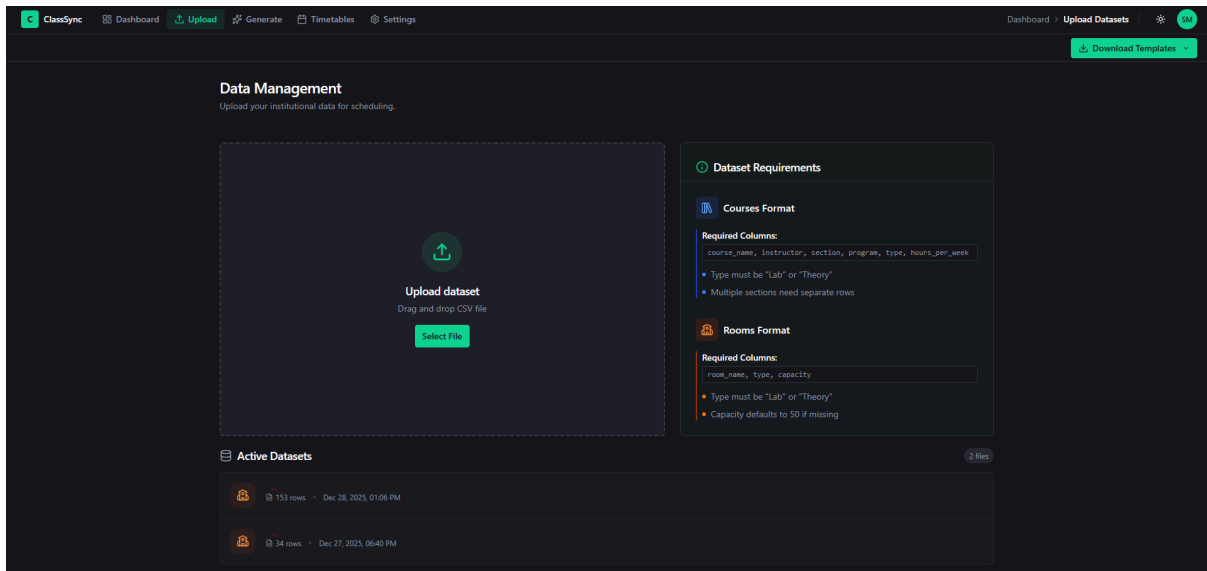
Prototype Screenshots

Prototype: <https://classsync-ai.up.railway.app/>

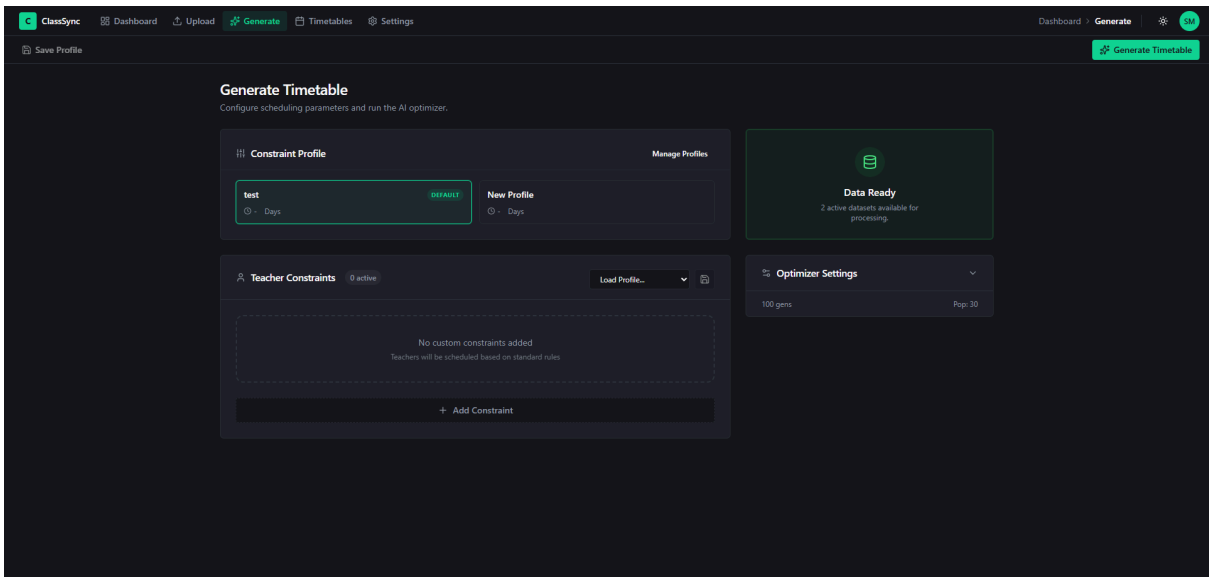
Dashboard



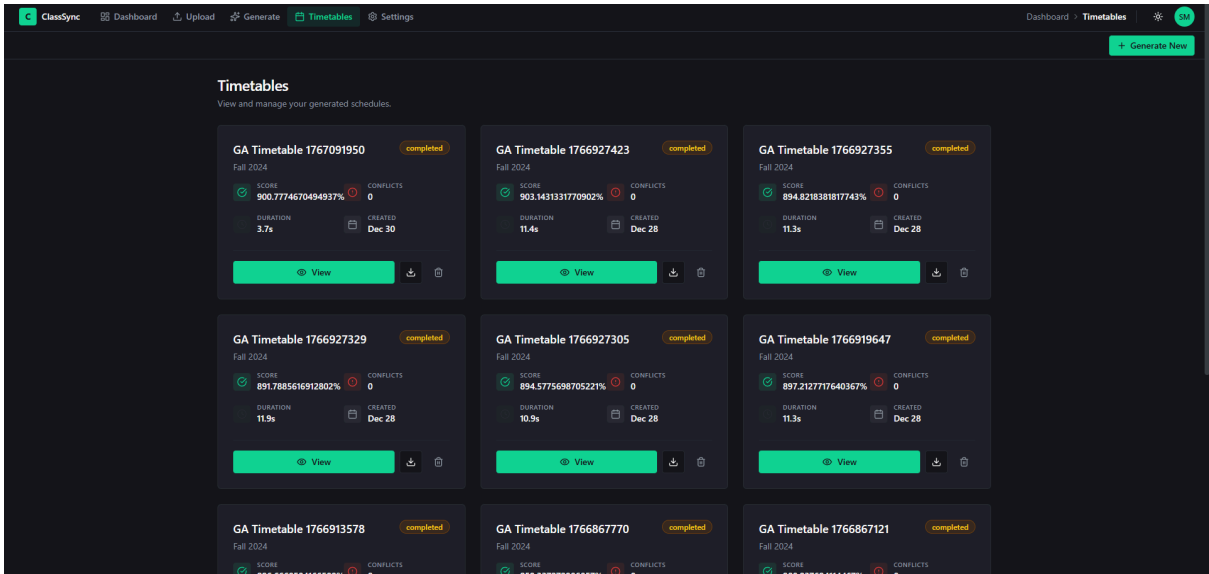
Upload



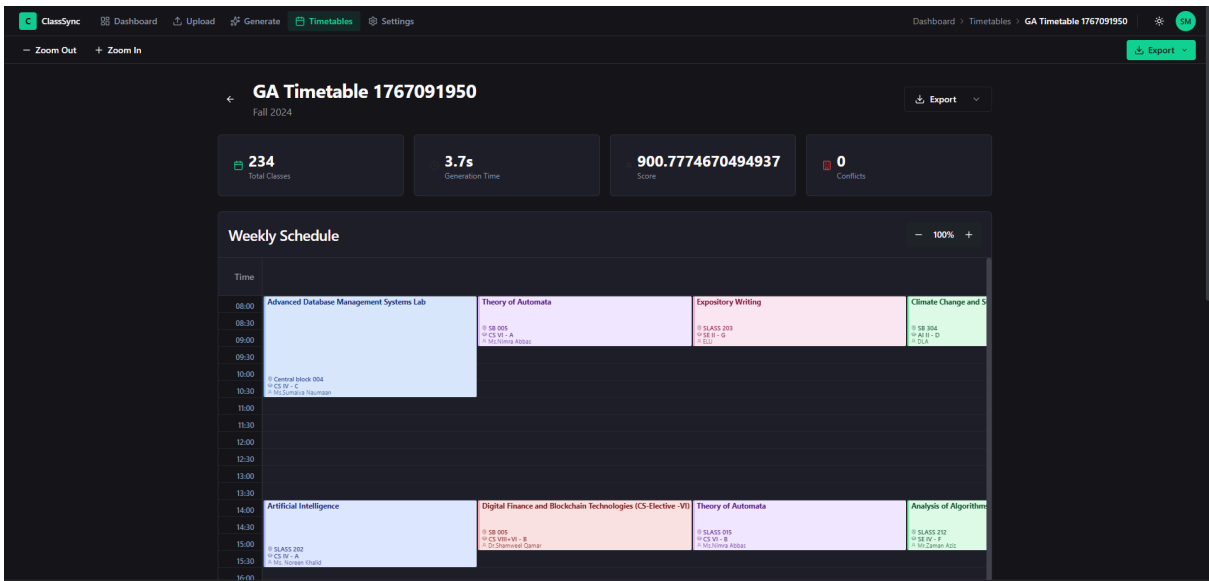
Generate



Timetables



View Timetable



Settings

Settings
Configure scheduling constraints and algorithm parameters.

Constraint Profiles

- test (ACTIVE)
- New Profile

test
System Default Profile

Schedule Days: m | Slot Duration: m | Start Time: | End Time: |

HARD RULES | PREFERENCES

Algorithm Engine Parameters

Population Size (Mock): 30 | Generations (Mock): 100 | Max Time (s): 300

Save Config

Input

AutoSave OFF

Room...

Search

File Home Insert Draw Page Layout Formulas Data Review View Automate Help Acrobat Comments Share

Clipboard

Font

Paragraph

Number

Conditional Formatting

Cells

Editing

Add-ins

Create PDF

Address Acrobat

POSSIBLE DATA LOSS

Some features might be lost if you save this workbook in the comma-delimited (.csv) format. To preserve these features, select it in an Excel file format.

Don't show again Save As...

M41

1

A

B

C

D

E

F

G

H

I

J

K

L

M

1	Rooms	Type											
2	Central block 002	Lab											
3	Central block 003	Lab											
4	Central block 004	Lab											
5	Central block 105	Lab											
6	SB 003 LAB	Lab											
7	SB 004	Theory											
8	SB 005	Theory											
9	SB 205	Theory											
10	SB 206	Theory											
11	SB 301	Theory											
12	SB 302	Theory											
13	SB 304	Theory											
14	SB 306	Theory											
15	SB 307	Theory											
16	SB AUD 001	Theory											
17	SLASS 013	Theory											
18	SLASS 015	Theory											
19	SLASS 118	Theory											
20	SLASS 201	Theory											
21	SLASS 202	Theory											
22	SLASS 203	Theory											
23	SLASS 204	Theory											
24	SLASS 206	Theory											
25	SLASS 207	Theory											
26	SLASS 208	Theory											
27	SLASS 212	Theory											
28	SLASS 309	Theory											
29	SLASS B002	Lab											
30	SLASS SMC 001 Lab	Lab											
31	SBT 202	Theory											
32	SBT 204	Theory											
33	SBT 303	Theory											
34	SBT 305	Theory											
35	SVAD 403 Lab												

Rooms

+

100%

AutoSave OFF

Course...

Search

File Home Insert Draw Page Layout Formulas Data Review View Automate Help Acrobat Comments Share

Clipboard

Font

Paragraph

Number

Conditional Formatting

Cells

Editing

Add-ins

Create PDF

Address Acrobat

POSSIBLE DATA LOSS

Some features might be lost if you save this workbook in the comma-delimited (.csv) format. To preserve these features, select it in an Excel file format.

Don't show again Save As...

A1

1

A

B

C

D

E

F

G

H

I

1	Course Name	Hours per week	Program	Section	Instructor	Type			
2	Advanced Database Management Systems	2	CS4	A	Ms. Talia Noreen	Theory			
3	Advanced Database Management Systems	2	CS4	B	Ms. Talia Noreen	Theory			
4	Advanced Database Management Systems	2	CS4	C	Ms. Talia Noreen	Theory			
5	Advanced Database Management Systems Lab	3	CS4	A	Mr. Ahsan Atiq	Lab			
6	Advanced Database Management Systems Lab	3	CS4	B	Mr. Ahsan Atiq	Lab			
7	Advanced Database Management Systems Lab	3	CS4	C	Mr. Ahsan Atiq	Lab			
8	Applied Physics	2	SE2	A	Mr. Uzair Bashir	Theory			
9	Applied Physics	2	SE2	B	Mr. Uzair Bashir	Theory			
10	Applied Physics Lab	3	SE2	A	Ms. Amna Rafiq	Lab			
11	Applied Physics Lab	3	SE2	B	Ms. Amna Rafiq	Lab			
12	Artificial Intelligence	2	CS4	A	Dr. Usman Nazir	Theory			
13	Artificial Intelligence	2	CS4	B	Ms. Noreen Khalid	Theory			
14	Artificial Intelligence	2	CS4	C	Ms. Noreen Khalid	Theory			
15	Artificial Intelligence	2	CS6	A	Dr. Usman Nazir	Theory			
16	Artificial Intelligence	2	CS6	B	Mr. Saad Azhar Saeed	Theory			
17	Artificial Intelligence Lab	3	CS4	A	Mr. Osama Tariq	Lab			
18	Artificial Intelligence Lab	3	CS4	B	Mr. Osama Tariq	Lab			
19	Artificial Intelligence Lab	3	CS4	C	Ms. Noreen Khalid	Lab			
20	Artificial Intelligence Lab	3	CS6	A	Mr. Osama Tariq	Lab			
21	Artificial Intelligence Lab	3	CS6	B	Mr. Osama Tariq	Lab			
22	Business Computing	3	MB02	-	Ms. Sheeza Batool	Theory			
23	Business Computing Lab	3	MB02	-	Ms. Talia Noreen	Lab			
24	Business Process Engineering	3	SE8	-	SMS Faculty	Theory			
25	Computer Networks	2	CS6	A+B	Mr. Nabeel Ahsan	Theory			
26	Computer Networks	2	MB04	-	Mr. Muhammad Ali	Theory			
27	Computer Networks Lab	3	CS6	B	Mr. Shazil Ali	Lab			
28	Computer Networks Lab	3	CS6	A	Mr. Shazil Ali	Lab			
29	Computer Organization & Assembly Language	2	CS4	A	Ms. Nimra Abbas	Theory			
30	Computer Organization & Assembly Language	2	CS4	B	Ms. Nimra Abbas	Theory			
31	Computer Organization & Assembly Language	2	CS4	C	Ms. Nimra Abbas	Theory			
32	Computer Organization & Assembly Language Lab	3	CS4	B	Ms. Ayesha Seher	Lab			
33	Computer Organization & Assembly Language Lab	3	CS4	C	Ms. Ayesha Seher	Lab			
34	Computer Organization & Assembly Language Lab	3	CS4	A	Ms. Nimra Abbas	Lab			
35	Data Structures & Algorithms	3	CS4	A	Ms. Huda Sarfraz	Theory			

Courses

+

100%

Output

AutoSave OFF

GA_Timetable_1760743906_master.xlsx

Protected View

Search

File Home Insert Draw Page Layout Formulas Data Review View Automate Help Acrobat

Clipboard

Font

Paragraph

Number

Conditional Formatting

Cells

Editing

Add-ins

Create PDF

Address Acrobat

PROTECTED VIEW

Be careful—files from the Internet can contain viruses. Unless you need to edit, it's safer to stay in Protected View.

Enable Editing

A1

1

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

1																				
2																				
3	Time	Course	Program	Section	Instructor	Room	Building	Type	Duration											
4	08:00 - 09:30	Linear Algebra	CS4	C	Mr. Farhan Ali	SLASS 212	N/A	lecture_hall	90 min											
5	08:00 - 09:30	Data Structures & Algorithms	CS4	B	Ms. Hamna Anwar	SB 307	N/A	lecture_hall	90 min											
6	08:00 - 11:00	Artificial Intelligence Lab	CS6	A-1	Mr. Osama Tariq	SLASS SMC 001 Lab	N/A	lab	180 min											
7	08:00 - 11:00	Object Oriented Programming Lab	SE2	E	Mr. Ameer Hamza	SVAD 403 Lab	N/A	lab	180 min											
8	09:30 - 11:00	Data Structures & Algorithms	CS4	A	Ms. Huda Sarfraz	SBT 202	N/A	lecture_hall	90 min											
9	09:30 - 11:00	Business Process Engineering	SE8	-	SMS Faculty	SLASS 202	N/A	lecture_hall	90 min											
10	09:30 - 11:30	Computer Networks	MB04	-	Mr. Muhammad Ali	SBT 305	N/A	lecture_hall	120 min											
11	09:30 - 11:30	Digital Logic Design	CS2	C	Ms. Arooma Amir	SB 304	N/A	lecture_hall	120 min											
12	11:00 - 12:30	Probability for Data Science	MB02	A	Ms. Amna Humayun	SB 005	N/A	lecture_hall	90 min											
13	11:00 - 12:30	Business Computing	MB02	-	Ms. Sheeza Batool	SB 004	N/A	lecture_hall	90 min											
14	14:00 - 15:30	Probability & Statistics	CS2	B	Ms. Amna Humayun	SB 005	N/A	lecture_hall	90 min											
14	14:00 - 15:30	Probability & Statistics	CS2	C	Dr. Aamir Majeed Chaudhary	SBT 202	N/A	lecture_hall	90 min											
14	14:00 - 15:30	Multivariable Calculus	CS2	D	Mr. Ali Haider	SBT 204	N/A	lecture_hall	90 min											
14	14:00 - 15:30	Probability & Statistics	SE2	F	Mr. Hamza Sarfraz	SB 301	N/A	lecture_hall	90 min											
14	14:00 - 15:30	Media Entrepreneurship	SE8	-	SMC Jsu	SLASS 207	N/A	lecture_hall	90 min											
14	14:00 - 15:30	Object Oriented Programming	CS2	A	Mr. Nouman Ali	SBT 305	N/A	lecture_hall	90 min											
14	14:00 - 17:00	Advanced Database Management Systems Lab	CS4	C	Mr. Ahsan Atiq	SLASS SMC 001 Lab	N/A	lab	180 min											
20	14:00 - 15:30	Theory of Programming Languages	CS8	B	Ms. Huda Sarfraz	SLASS 202	N/A	lecture_hall	90 min											
21	14:00 - 15:30	Expository Writing	MB02	A	Ms. Fatima Bilal	SLASS 013	N/A	lecture_hall	90 min											
22	14:00 - 16:00	Advanced Database Management Systems	CS4	C	Ms. Talia Noreen	SB 302	N/A	lecture_hall	120 min											
23	14:00 - 15:30	Management Information Systems	MB04	-	Dr. Natasha Ali	SLASS 204	N/A	lecture_hall	90 min											
23	15:30 - 17:00	Data Structures & Algorithms	CS4	B	Ms. Hamna Anwar	SLASS 203	N/A	lecture_hall	90 min											
23	15:30 - 17:00	Multivariable Calculus	CS2	A	Dr. Zavar Hussain VF	SBT 202	N/A	lecture_hall	90 min											
23	15:30 - 17:30	Applied Physics	SE2	A	Mr. Uzair Bashir	SB AUD 001	N/A	lecture_hall	120 min											
23	15:30 - 17:30	Object Oriented Programming	CS2	C	Mr. Asim Inshad	SLASS 206	N/A	lecture_hall	90 min											
28	15:30 - 17:00	Probability & Statistics	CS2	D	Dr. Aamir Majeed Chaudhary	SLASS 204	N/A	lecture_hall	90 min											
29	15:30 - 18:30	Artificial Intelligence Lab	CS6	B-1	Mr. Osama Tariq	Central block 105	N/A	lab	180 min											
30	15:30 - 18:30	Artificial Intelligence Lab	CS4	C	Ms. Noreen Khalid	SB 003 LAB	N/A	lab	180 min											
31	15:30 - 17:00	Probability & Statistics	SE2	E	Mr. Hamza Sarfraz	SLASS 015	N/A	lecture_hall	90 min											
33	15:30 - 17:00	Immersive Media	CS+SE8	A	SMC Jsu	SB 306	N/A	lecture_hall	90 min											
33	17:00 - 18:30	Object Oriented Programming	SE2	E	Mr. Nabeel Ahsan	SB 004	N/A	lecture_hall	90 min											
34	17:00 - 18:30	Probability & Statistics	CS2	C	Dr. Aamir Majeed Chaudhary	SLASS 212	N/A	lecture_hall	90 min											

Master Timetable

+

100%

Testing

Test Cases

TC-01: Verify Login Functionality

- **Test Case Description:** Testing if the system authenticates a user with valid credentials.
- **Test Prerequisite:**
 - The system database is online.
 - A valid user account exists (e.g., admin@domain.com).
- **Test Inputs:**
 - Email: admin@domain.com
 - Password: Secure Password
- **Test Steps:**
 - Navigate to the Landing Page.
 - Click on the "Login" button.
 - Enter the valid Email and Password.
 - Click "Submit".
- **Expected Results:**
 - The system verifies the credentials.
 - A secure JWT session is established.
 - The user is redirected to the main Dashboard.

TC-02: Verify Dataset Upload

- **Test Case Description:** Testing if the uploading functionality works for valid CSV files.
- **Test Prerequisite:**
 - The user is logged in as Coordinator.
 - Valid "Supervised data" (CSV files) is available on the device.
- **Test Inputs:**
 - Courses.csv
 - Rooms.csv
- **Test Steps:**
 - Navigate to the "Upload Data" Tab.
 - Click on the "Select File" button.
 - Select the valid Courses.csv and Rooms.csv files.
 - Click the "Upload" button.
- **Expected Results:**
 - The System validates the data schema.
 - The System saves the data to the cloud.
 - A "Dataset Validated" success message is displayed.

TC-03: Verify Data Validation Logic

- **Test Case Description:** Testing if the system correctly identifies and accepts schema-compliant data.
- **Test Prerequisite:**
 - A file upload process has been initiated (TC-02).
- **Test Inputs:**
 - A valid CSV file with correct headers and data types.
- **Test Steps:**
 - Initiate the upload of a valid file.
 - Wait for the automatic validation process to complete.
- **Expected Results:**
 - The system parses the file without errors.
 - The system checks for duplicates and logical inconsistencies.
 - The validation status indicator turns Green (Valid).

TC-04: Verify Constraint Configuration

- **Test Case Description:** Testing if the user can successfully modify and save hard/soft constraints.
- **Test Prerequisite:**
 - The user is logged in as Admin.
- **Test Inputs:**
 - Hard Constraint: "Enforce Room Capacity" (Set to True)
 - Soft Constraint: "Minimize Gaps" (Set to High)
- **Test Steps:**
 - Navigate to the "settings" configuration panel.
 - Click on "edit details".
 - Change values for the constraints
 - Option to turn the predefined checkboxes to on or off
 - The system verifies the constraints
 - Click the "Save" button.
- **Expected Results:**
 - The system persists the new configuration to the database.
 - A confirmation message "Constraints Updated" is displayed.

TC-05: Verify Preference Settings

- **Test Case Description:** Testing if global scheduling preferences (weights) can be adjusted.
- **Test Prerequisite:**
 - The user is accessing the Constraints panel.
- **Test Inputs:**

- Global Setting: "Timeslot Duration" = 90 mins.
- **Test Steps:**
 - Navigate to "Global Preferences".
 - Select "90 minutes" from the Timeslot Duration dropdown.
 - Save the changes.
- **Expected Results:**
 - The system updates the scoring algorithm parameters.
 - Future generations will use 90-minute blocks.

TC-06: Verify Timetable Generation

- **Test Case Description:** Testing if the Genetic Algorithm produces a conflict-free schedule.
- **Test Prerequisite:**
 - Valid datasets are uploaded (TC-02).
 - Subscription is active.
- **Test Inputs:**
 - Trigger: "Generate Timetable" button.
- **Test Steps:**
 - Navigate to the "Generate" tab.
 - Click the "Generate Timetable" button.
 - Wait for the process (max 60 seconds).
- **Expected Results:**
 - The system executes the optimization engine.
 - A conflict-free timetable is generated.
 - The results are displayed on the screen.

TC-07: Verify View Timetable Filtering

- **Test Case Description:** Testing if the user can view the schedule filtered by specific criteria.
- **Test Prerequisite:**
 - A timetable has been generated (TC-06).
- **Test Inputs:**
 - Filter: "Room R-101"
- **Test Steps:**
 - Navigate to the Timetable Dashboard.
 - Select "Room View" from the dropdown.
 - Select "R-101".
- **Expected Results:**
 - The grid updates to show classes only for Room R-101.
 - The view renders correctly with color coding.

TC-08: Verify Timetable Export

- **Test Case Description:** Testing if the system exports the schedule to a downloadable file.
- **Test Prerequisite:**
 - A timetable is currently visible.
- **Test Inputs:**
 - Format: "PDF"
- **Test Steps:**
 - Click the "Export" button.
 - Select "PDF" format.
 - Confirm the download.
- **Expected Results:**
 - The system compiles the data.
 - A .pdf file is downloaded to the user's device.
 - The file content matches the displayed schedule.

TC-09: Verify AI Help Response

- **Test Case Description:** Testing if the AI Assistant provides relevant answers to user queries.
- **Test Prerequisite:**
 - The user is logged in.
- **Test Inputs:**
 - Query: "How do I upload data?"
- **Test Steps:**
 - Open the AI Chat sidebar.
 - Type "How do I upload data?".
 - Press Enter.
- **Expected Results:**
 - The AI processes the query.
 - The AI displays text instructions or navigates the user to the Upload tab.

TC-10: Verify What-If Scenario Simulation

- **Test Case Description:** Testing if a simulation runs without affecting the live schedule.
- **Test Prerequisite:**
 - A baseline schedule exists.
- **Test Inputs:**
 - Scenario Change: "Mark Lab-A as Unavailable"
- **Test Steps:**

- Enter "Simulation Mode".
- Apply the change "Mark Lab-A as Unavailable".
- Click "Run Simulation".
- **Expected Results:**
 - A temporary schedule is generated.
 - A "Comparison View" appears showing the Original vs. New schedule.
 - The live database remains unchanged.

TC-11: Verify Schedule Explanation

- **Test Case Description:** Testing if the AI can explain the logic behind a specific slot assignment.
- **Test Prerequisite:**
 - A timetable is visible.
- **Test Inputs:**
 - Selection: "CS101 at 9:00 AM"
- **Test Steps:**
 - Click on the "CS101" session block.
 - Click the "Explain this slot" button.
- **Expected Results:**
 - The AI analyzes the constraints (e.g., Room availability).
 - The AI displays a reason (e.g., "Placed here because Instructor is only available 9-11 AM").

Project Management

Open Issues

There is no REAL Correct TimeTable as every faculty has their own constraints and you cannot have fully optimized results where everyone is content with the final output. There will always be overlapping constraints. One can never achieve an optimal solution as there is no way to define an optimal solution.

Project Evaluation

Learning Outcomes

Working on ClassSync AI taught us so much more than we expected. The biggest technical skill we gained was understanding how genetic algorithms actually work in practice. We spent hours tweaking parameters and eventually figured out that adaptive mutation works best - when fitness stops improving, increase mutation to explore more options.

Database design with PostgreSQL was challenging but rewarding. Implementing multi-tenant architecture taught us about data security and keeping different universities' data completely separate. We learned to denormalize certain fields to make queries faster when the genetic algorithm checks constraints hundreds of times per second.

The problem-solving aspect was where we grew the most. Timetable scheduling is an NP-hard problem with no guaranteed perfect solution. We learned to distinguish between hard constraints (absolute rules like "a teacher can't be in two places at once") and soft constraints (preferences like "minimize student gaps"), and created a penalty system to guide the algorithm toward good solutions.

We actually started working on ClassSync AI last semester with a completely build-and-fix approach - just write code, see what breaks, patch it, and move on. There was no planning, no requirements documentation, nothing. After studying Software Engineering this semester and learning about proper process models and system design, everything clicked. We realized what our system was actually lacking and rebuilt it properly using the Rapid Prototyping Model. Our system has completely transformed into a better, more reliable shape with structured architecture, proper constraint management, and systematic validation.

Overall Experience

This project was one of the most challenging things we've done, but also the most rewarding. The biggest struggle was getting the genetic algorithm to behave consistently - sometimes it would generate perfect schedules in seconds, other times it would get stuck with unresolvable conflicts. Working as a team of three taught us valuable collaboration skills - Saad handled algorithm optimization, Ibrahim led frontend development, and I focused on database architecture. We had disagreements about technology choices and priorities, but learned to resolve them through testing and prototyping rather than endless arguments. Looking back, we made mistakes like delaying automated testing, but the Rapid Prototyping Model saved us by revealing issues early.

What made this project truly meaningful was knowing it solves a real problem. Our coordinator spends 80-100 hours every semester manually creating timetables, dealing with the "butterfly effect" where changing one class cascades into dozens of conflicts. This system addresses this situation comprehensively.

The project taught us that there's no "perfect" timetable - just the best compromise between competing constraints - and that software engineering is about making informed trade-offs.

Conclusion

ClassSync AI transformed from a chaotic build-and-fix project into a fully functional, deployable system through the systematic application of software engineering principles. This journey demonstrated that successful software development requires proper requirements analysis, structured system design, and iterative prototyping rather than just writing code. The contrast between our unstructured approach last semester and our structured methodology this semester proved the value of following established software engineering practices.

The system successfully addresses the real problem of manual timetable creation, reducing 80-100 hours of work to 3-4 seconds of automated scheduling. Through implementing genetic algorithms, multi-tenant database architecture, and full-stack development with FastAPI and React, we gained practical experience with industry-standard technologies and methodologies.